



Online

Object Oriented Programming

Computer Systems Engineering

José Manuel Rodríguez Cantú

00612676

Activity 3: Class hierarchy design for a hospital software system

Module 3: Inheritance and Polymorphism

José Abdón Espínola González

Date:

July 24, 2026

1. Description of the solution

This activity models the different people found in a regional hospital with an object-oriented class hierarchy. The design rests on the three ideas the activity asks for: inheritance, abstraction and overloading. There are five files: the abstract class **Person**, its three subclasses **Doctor**, **Patient** and **Guard**, and a test program that creates a doctor, a patient and two guards, then registers each one.

Person is an abstract superclass. It stores the two data fields every person shares, **name** and **age**, and exposes them through *getName* and *getAge*. It also declares an abstract method, *register*, with no body. Because the class is abstract it cannot be instantiated on its own; it only serves as the common base for the specific people. That is abstraction: the shared idea of a person lives in one place and each subclass fills in the rest.

Doctor, **Patient** and **Guard** extend **Person**, so they inherit name, age and their getters. Each subclass adds only what makes it different: Doctor adds a department, Patient adds an illness, and Guard adds a shift and an optional phone. Each subclass reuses the **Person** constructor through *super(name, age)*, and the Guard's short constructor reaches it by chaining, so the shared data is set in one place instead of being repeated in each class.

Each subclass overrides *register* with its own version. When *register* is called, Java runs the version that belongs to the real object, so a Doctor prints "Welcome Doctor!", a Patient prints "Welcome Patient!" and a Guard prints "Welcome Guard!". This matches the learning goal: different people performing the same action in their own way, which in Java is called polymorphism.

Guard shows overloading with two constructors: one receives a phone number and the other does not. The shorter constructor calls the longer one with *this(name, age, shift, null)*, so the setup code is written only once and a guard can be created either way. The test uses John, created without a phone, and Kevin, created with one, to exercise both constructors.

2. Source code

Person.java

```

/**
 * Person.java
 * Abstract superclass for every person in the hospital system.
 * It keeps the data and behavior shared by doctors, patients and guards,
 * so that common code lives in a single place (abstraction + inheritance).
 * Object Oriented Programming - Module 3, Activity 3.
 */
public abstract class Person {
    // Data fields shared by every person
    private String name; // the person's name
    private int age;      // the person's age in years

    /** Constructor that sets the name and age common to every person. */
    public Person(String nameP, int ageP) {
        this.name = nameP;
        this.age = ageP;
    }

    /** Returns the person's name. */
    public String getName() {
        return name;
    }

    /** Returns the person's age. */
    public int getAge() {
        return age;
    }

    /**
     * Registers the person in the system. Each subclass greets its own kind
     * of person in its own way, so the method is abstract and every subclass
     * must provide its own version.
     */
    public abstract void register();
}

```

Doctor.java

```

/**
 * Doctor.java
 * A doctor is a Person who also belongs to a medical department.
 * Object Oriented Programming - Module 3, Activity 3.
 */
public class Doctor extends Person {
    private String department; // the department the doctor works in

    /** Creates a doctor with a name, an age and a department. */
    public Doctor(String nameD, int ageD, String departmentD) {
        super(nameD, ageD); // reuse the Person constructor for the shared data
        this.department = departmentD;
    }

    /** Returns the doctor's department. */
    public String getDepartment() {
        return department;
    }

    /** Greets the doctor when registering. */
    @Override
    public void register() {
        System.out.println("Welcome Doctor!");
    }
}

```

Patient.java

```

/**
 * Patient.java
 * A patient is a Person who is admitted with a certain illness.
 * Object Oriented Programming - Module 3, Activity 3.
 */
public class Patient extends Person {
    private String illness; // the illness the patient is treated for

    /** Creates a patient with a name, an age and an illness. */
    public Patient(String nameP, int ageP, String illnessP) {
        super(nameP, ageP); // reuse the Person constructor for the shared data
        this.illness = illnessP;
    }

    /** Returns the patient's illness. */
    public String getIllness() {
        return illness;
    }

    /** Greets the patient when registering. */
    @Override
    public void register() {
        System.out.println("Welcome Patient!");
    }
}

```

Guard.java

```

/**
 * Guard.java
 * A guard is a Person who covers a shift and may have a contact phone.
 * This class shows constructor overloading: a guard can be created with or
 * without a phone number, and the shorter constructor reuses the longer one.
 * Object Oriented Programming - Module 3, Activity 3.
 */
public class Guard extends Person {
    private String shift; // the shift the guard covers (for example, Morning)
    private String phone; // the guard's contact phone (optional)

    /**
     * Creates a guard without a phone number. It calls the other constructor
     * with null as the phone, so the setup code is written only once.
     */
    public Guard(String nameG, int ageG, String shiftG) {
        this(nameG, ageG, shiftG, null); // constructor chaining (overloading)
    }

    /** Creates a guard with a phone number. */
    public Guard(String nameG, int ageG, String shiftG, String phoneG) {
        super(nameG, ageG); // reuse the Person constructor for the shared data
        this.shift = shiftG;
        this.phone = phoneG;
    }

    /** Returns the guard's shift. */
    public String getShift() {
        return shift;
    }

    /** Returns the guard's phone, or null if none was provided. */
    public String getPhone() {
        return phone;
    }

    /** Greets the guard when registering. */
    @Override
    public void register() {
        System.out.println("Welcome Guard!");
    }
}

```

HospitalTest.java

```

/**
 * HospitalTest.java
 * Test program for the hospital class hierarchy. It creates the four people
 * requested in the activity and registers each one. The Guard cases show the
 * two overloaded constructors: John is created without a phone and Kevin with
 * a phone.
 * Object Oriented Programming - Module 3, Activity 3.
 */
public class HospitalTest {
    public static void main(String[] args) {
        // Test data requested in the activity
        Doctor doctor = new Doctor("Joseph", 41, "Neurologist");
        Patient patient = new Patient("Richard", 78, "Chronic Headache");
        Guard guardNoPhone = new Guard("John", 39, "Morning"); //
        // 3-argument constructor
        Guard guardWithPhone = new Guard("Kevin", 43, "Afternoon", "+52 232 456345"); //
        // 4-argument constructor

        System.out.println("=== Hospital registration system ===");

        // Doctor
        System.out.println();
        System.out.println("Doctor");
        System.out.println("  Name: " + doctor.getName());
        System.out.println("  Age: " + doctor.getAge());
        System.out.println("  Department: " + doctor.getDepartment());
        doctor.register();

        // Patient
        System.out.println();
        System.out.println("Patient");
        System.out.println("  Name: " + patient.getName());
        System.out.println("  Age: " + patient.getAge());
        System.out.println("  Illness: " + patient.getIllness());
        patient.register();

        // Guard created without a phone (shorter overloaded constructor)
        System.out.println();
        System.out.println("Guard");
        System.out.println("  Name: " + guardNoPhone.getName());
        System.out.println("  Age: " + guardNoPhone.getAge());
        System.out.println("  Shift: " + guardNoPhone.getShift());
        System.out.println("  Phone: " + (guardNoPhone.getPhone() == null ? "not provided"
: guardNoPhone.getPhone()));
        guardNoPhone.register();

        // Guard created with a phone (longer overloaded constructor)
        System.out.println();
        System.out.println("Guard");
        System.out.println("  Name: " + guardWithPhone.getName());
        System.out.println("  Age: " + guardWithPhone.getAge());
        System.out.println("  Shift: " + guardWithPhone.getShift());
        System.out.println("  Phone: " + (guardWithPhone.getPhone() == null ? "not
provided" : guardWithPhone.getPhone()));
    }
}

```

```
        guardWithPhone.register();  
    }  
}
```

3. Execution evidence

The five files compile with `javac` without errors or warnings, and the test program prints the data of the four people together with the welcome message produced by each `register` method. John, built with the three-argument constructor, shows "not provided" for the phone, while Kevin, built with the four-argument constructor, displays his number. The terminal output below shows the compilation and the run:



```
$ javac Person.java Doctor.java Patient.java Guard.java HospitalTest.java  
$ java HospitalTest  
=== Hospital registration system ===  
  
Doctor  
  Name: Joseph  
  Age: 41  
  Department: Neurologist  
Welcome Doctor!  
  
Patient  
  Name: Richard  
  Age: 78  
  Illness: Chronic Headache  
Welcome Patient!  
  
Guard  
  Name: John  
  Age: 39  
  Shift: Morning  
  Phone: not provided  
Welcome Guard!  
  
Guard  
  Name: Kevin  
  Age: 43  
  Shift: Afternoon  
  Phone: +52 232 456345  
Welcome Guard!
```

Each person is registered through its own `register` method, so the same call produces a different greeting depending on the object. The two `Guard` objects confirm that both overloaded constructors

work: the one without a phone falls back to "not provided" and the one with a phone keeps the number.